

파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

머릿말

Note. 참고 URL

https://github.com/seejokim1/NA_with_python

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

Contents

1	파이썬의 이해	1
2	비선형 방정식 구하기	2
3	선형연립방정식 수치해법	3
4	수치미분 (Numerical Differentiation)	4
5	수치적분 (Numerical Integration)	5
6	수치 보간 (Interpolation)	6
7	수치 회귀 (Numerical Regression)	7
8	비선형연립방정식 수치해법	8
9	상미분 방정식 (초기치문제)	9
10	Shooting Method (경계치 문제)	10
11	유한차분법	11
11.1	편미분 방정식 개요	11
11.1.1	편미분 방정식 종류	11
11.2	유한차분법의 수치해석	11
11.2.1	유한차분법(FDM) 개요	11
11.2.2	1차원 비정상 열전달(확산) 방정식의 유한차분법	12
11.2.3	Explicit Euler Method (명시적 유한차분법)	14
11.2.4	Implicit Euler Method (암시적 유한차분법)	15
11.2.5	Crank–Nicolson Method (크랭크–니콜슨법)	16
11.2.6	기타 중요 사항	18
11.2.7	1차원 정상 열전달 FDM 수치해석	18
11.2.8	1차원 비정상 열전달 FDM 수치해석	19
11.3	연습문제	21

Chapter 1

파이썬의 이해

Chapter 2

비선형 방정식 구하기

Chapter 3

선형연립방정식 수치해법

Chapter 4

수치미분 (Numerical Differentiation)

Chapter 5

수치적분 (Numerical Integration)

Chapter 6

수치 보간 (Interpolation)

Chapter 7

수치 회귀 (Numerical Regression)

Chapter 8

비선형연립방정식 수치해법

Chapter 9

상미분 방정식 (초기치문제)

Chapter 10

Shooting Method (경계치 문제)

Chapter 11

유한차분법

11.1 편미분 방정식 개요

11.1.1 편미분 방정식 종류

대표적인 2차 선형 PDE:

$$Au_{xx} + Bu_{xy} + Cu_{yy} = 0$$

판별식:

$$B^2 - 4AC$$

- Elliptic: $B^2 - 4AC < 0$
- Parabolic: $B^2 - 4AC = 0$
- Hyperbolic: $B^2 - 4AC > 0$

대표 방정식:

- Laplace equation
- Heat equation
- Wave equation

11.2 유한차분법의 수치해석

11.2.1 유한차분법(FDM) 개요

1. 기본 개념

유한차분법(Finite Difference Method, FDM)은 미분방정식을 격자(grid) 위에서 차분(차이) 형태로 근사하여 대수방정식 시스템으로 변환하는 수치해석 기법이다.

Taylor 전개를 기반으로 미분항을 차분식으로 근사한다.

2. 대표적인 지배방정식 (Advection-Diffusion)

$$\frac{\partial u}{\partial t} + v_x \frac{\partial u}{\partial x} + v_y \frac{\partial u}{\partial y} = D \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + P(x, y) \quad (11.1)$$

- 첫 항: 시간 변화율
- 두 번째 항: 대류(Advection)
- 세 번째 항: 확산(Diffusion)
- 마지막 항: 생성항(Source term)

3. 유한차분 근사

시간 전진 차분:

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t}$$

공간 중앙 차분:

$$\frac{\partial u}{\partial x} \approx \frac{u_{i+1,j}^n - u_{i-1,j}^n}{2\Delta x}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2}$$

4. 이산화된 방정식 예

$$\frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t} + v_x \frac{u_{i+1,j}^n - u_{i-1,j}^n}{2\Delta x} + v_y \frac{u_{i,j+1}^n - u_{i,j-1}^n}{2\Delta y} \quad (11.2)$$

$$= D \left(\frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} \right) + P_{i,j}^n \quad (11.3)$$

5. 핵심 특징

- 미분방정식 $\rightarrow$ 선형 대수방정식으로 변환
- 격자 기반 계산
- 시간-공간 이산화 필요
- 안정성 조건 (CFL 조건 등) 고려 필요

11.2.2 1차원 비정상 열전달(확산) 방정식의 유한차분법

1. 지배 방정식 1차원 비정상 열전달 방정식:

$$\frac{\partial T}{\partial t} = D \frac{\partial^2 T}{\partial x^2}, \quad a < x < b, \quad t > 0 \quad (11.4)$$

2. 초기 및 경계 조건

$$T(x, 0) = T_0(x) \quad (11.5)$$

$$T(a, t) = T_a(t), \quad T(b, t) = T_b(t) \quad (11.6)$$

3. 시간 및 공간 이산화 (1) 시간 전진 차분 (Forward Difference)

$$\frac{\partial T}{\partial t} \approx \frac{T_i^{n+1} - T_i^n}{\Delta t} \quad (11.7)$$

(2) 공간 중앙 차분 (Central Difference)

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2} \quad (11.8)$$

4. Explicit Euler (FTCS) 스킴

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = D \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2} \quad (11.9)$$

정리하면,

$$T_i^{n+1} = T_i^n + \lambda (T_{i+1}^n - 2T_i^n + T_{i-1}^n), \quad \lambda = \frac{D\Delta t}{\Delta x^2} \quad (11.10)$$

5. Implicit Euler (BTCS) 스킴

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = D \frac{T_{i+1}^{n+1} - 2T_i^{n+1} + T_{i-1}^{n+1}}{\Delta x^2} \quad (11.11)$$

→ 시간 단계마다 선형 연립방정식 풀이 필요

6. 벡터 형태

$$\mathbf{T}^n = [T_1^n \quad T_2^n \quad \cdots \quad T_N^n]^T \quad (11.12)$$

Explicit:

$$\mathbf{T}^{n+1} = \mathbf{A}\mathbf{T}^n$$

Implicit:

$$\mathbf{A}\mathbf{T}^{n+1} = \mathbf{T}^n$$

핵심 정리

- PDE → 시간/공간 격자 이산화
- 중앙차분: 공간 2차 정확도
- Explicit: 계산 간단, 안정성 조건 필요
- Implicit: 안정성 우수, 행렬 풀이 필요
- 열확산 문제는 전형적인 Parabolic PDE

11.2.3 Explicit Euler Method (명시적 유한차분법)

주어진 1차원 열전도 방정식

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

시간에 대해 전진 차분, 공간에 대해 중앙 차분을 적용하면

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}$$

따라서 Explicit Euler 차분식은

$$u_i^{n+1} = u_i^n + r (u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

여기서 안정성 계수는

$$r = \frac{\alpha \Delta t}{\Delta x^2}$$

—

안정 조건 Von Neumann 안정성 해석 결과,

$$r \leq \frac{1}{2}$$

—

행렬 표현 벡터 형태로 표현하면

$$\mathbf{u}^{n+1} = [B]\mathbf{u}^n$$

여기서

$$[B] = \begin{bmatrix} 1-2r & r & 0 & \cdots & 0 \\ r & 1-2r & r & \cdots & 0 \\ 0 & r & 1-2r & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & r \\ 0 & 0 & 0 & r & 1-2r \end{bmatrix}$$

—

Python 구현 예제

```
def heat_explicit(u, r):
    u_new = u.copy()
    for i in range(1, len(u)-1):
        u_new[i] = u[i] + r*(u[i+1] - 2*u[i] + u[i-1])
    return u_new
```

11.2.4 Implicit Euler Method (암시적 유한차분법)

중앙차분을 사용하므로 공간 정확도는

$$O(\Delta x^2)$$

이다.

Implicit Euler 방법은 시간에 대해 후진 차분을 사용한다.

유한차분 근사

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2}$$

안정성 계수는

$$r = \frac{\alpha \Delta t}{\Delta x^2}$$

차분식 정리 정리하면

$$-ru_{i-1}^{n+1} + (1 + 2r)u_i^{n+1} - ru_{i+1}^{n+1} = u_i^n, \quad i = 1, \dots, N$$

행렬 표현 벡터 형태로 표현하면

$$[A]\mathbf{u}^{n+1} = \mathbf{u}^n$$

여기서 계수 행렬 $[A]$ 는

$$[A] = \begin{bmatrix} 1+2r & -r & 0 & \cdots & 0 \\ -r & 1+2r & -r & \cdots & 0 \\ 0 & -r & 1+2r & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -r \\ 0 & 0 & 0 & -r & 1+2r \end{bmatrix}$$

특징

- 시간: Backward Euler
- 공간: Central Difference
- 무조건 안정 (Unconditionally Stable)
- 매 시간 단계마다 선형 연립방정식 풀이 필요

Python 구현 예제

```
import numpy as np

def heat_implicit(u, r):
    N = len(u)
    A = np.zeros((N, N))

    for i in range(1, N-1):
        A[i, i-1] = -r
        A[i, i] = 1 + 2*r
        A[i, i+1] = -r

    A[0,0] = 1
    A[-1,-1] = 1

    return np.linalg.solve(A, u)
```

11.2.5 Crank–Nicolson Method (크랭크–니콜슨법)

Crank–Nicolson 방법은 Explicit 방법과 Implicit 방법의 평균이다.

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

시간에 대해 중앙 평균을 적용하면

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{\alpha}{2} \left(\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} + \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} \right)$$

안정성 계수

$$r = \frac{\alpha \Delta t}{\Delta x^2}$$

차분식 정리 정리하면

$$-\frac{r}{2}u_{i-1}^{n+1} + (1+r)u_i^{n+1} - \frac{r}{2}u_{i+1}^{n+1} = \frac{r}{2}u_{i-1}^n + (1-r)u_i^n + \frac{r}{2}u_{i+1}^n$$

행렬 표현 벡터 형태로 표현하면

$$[A]\mathbf{u}^{n+1} = [B]\mathbf{u}^n$$

여기서

$$[A] = \begin{bmatrix} 1+r & -\frac{r}{2} & 0 & \cdots & 0 \\ -\frac{r}{2} & 1+r & -\frac{r}{2} & \cdots & 0 \\ 0 & -\frac{r}{2} & 1+r & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -\frac{r}{2} \\ 0 & 0 & 0 & -\frac{r}{2} & 1+r \end{bmatrix}$$

$$[B] = \begin{bmatrix} 1-r & \frac{r}{2} & 0 & \cdots & 0 \\ \frac{r}{2} & 1-r & \frac{r}{2} & \cdots & 0 \\ 0 & \frac{r}{2} & 1-r & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \frac{r}{2} \\ 0 & 0 & 0 & \frac{r}{2} & 1-r \end{bmatrix}$$

특징

- 시간 정확도: $O(\Delta t^2)$
- 공간 정확도: $O(\Delta x^2)$
- 무조건 안정 (Unconditionally Stable)
- 매 시간 단계마다 선형 연립방정식 풀이 필요

Python 구현 예제

```
import numpy as np

def heat_CN(u, r):
    N = len(u)
    A = np.zeros((N, N))
    B = np.zeros((N, N))

    for i in range(1, N-1):
        A[i,i-1] = -r/2
        A[i,i]   = 1 + r
        A[i,i+1] = -r/2

        B[i,i-1] = r/2
        B[i,i]   = 1 - r
        B[i,i+1] = r/2

    A[0,0] = A[-1,-1] = 1
    B[0,0] = B[-1,-1] = 1

    return np.linalg.solve(A, B @ u)
```

11.2.6 기타 중요 사항

- CFL 조건
- 안정성 분석 (Von Neumann stability)
- 일관성(consistency)
- 수렴성(convergence)

11.2.7 1차원 정상 열전달 FDM 수치해석

1차원 정상 열전도 방정식

$$\frac{d^2 u(x)}{dx^2} = -\varepsilon, \quad x \in [0, 1]$$

비정상 항이 없으므로 시간항은 제거된다.

유한차분 근사 중앙 차분을 적용하면

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} = -\varepsilon$$

정리하면

$$-u_{i-1} + 2u_i - u_{i+1} = \varepsilon \Delta x^2$$

행렬 표현 벡터 형태로 표현하면

$$[A]\mathbf{u} = \mathbf{f}$$

여기서 계수 행렬은

$$[A] = \begin{bmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & -1 & 2 \end{bmatrix}$$

우변 벡터는

$$\mathbf{f} = \varepsilon \Delta x^2 \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

경계 조건 처리 Dirichlet 경계 조건:

$$u(0) = 1, \quad u(1) = e$$

코드에서는 다음과 같이 처리한다.

- 행렬의 첫 행과 마지막 행을 단위 행으로 변경
- 우변 벡터에 경계값 직접 대입

즉,

$$\begin{aligned} A[0, :] &= 0, & A[0, 0] &= 1 \\ A[-1, :] &= 0, & A[-1, -1] &= 1 \end{aligned}$$

$$f[0] = 1, \quad f[-1] = e$$

Python 구현 예제

```
import numpy as np
```

```
def steady_1D_FDM(N, eps):
    dx = 1.0/(N-1)
    A = np.zeros((N,N))
    f = np.ones(N)*eps*dx**2
```

```
    for i in range(1,N-1):
        A[i,i-1] = -1
        A[i,i] = 2
        A[i,i+1] = -1
```

```
    # Dirichlet BC
    A[0,:] = 0; A[0,0] = 1
    A[-1,:] = 0; A[-1,-1] = 1
    f[0] = 1
    f[-1] = np.e
```

```
    return np.linalg.solve(A,f)
```

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter11/section_11_02_07_1d_steady_heat_fdm.py

11.2.8 1차원 비정상 열전달 FDM 수치해석

1차원 비정상 열전도 방정식

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad x \in [0, 1], \quad t > 0$$

초기 조건과 경계 조건이 주어졌다고 가정한다.

수치 방법 선택 다음 세 가지 방법을 선택할 수 있다.

- 1: Explicit Euler (명시적)
- 2: Implicit Euler (암시적)
- 3: Crank–Nicolson

안정성 계수

$$r = \frac{\alpha \Delta t}{\Delta x^2}$$

- Explicit: $r \leq 0.5$ 조건 필요
- Implicit: 무조건 안정
- Crank–Nicolson: 무조건 안정, 2차 정확도

—

시간 단계별 해 구조 시간 단계 n 에서의 온도 벡터를

$$\mathbf{u}^n = [u_0^n \ u_1^n \ \cdots \ u_N^n]^T$$

로 정의한다.

Crank–Nicolson 방법의 경우

$$[A]\mathbf{u}^{n+1} = [B]\mathbf{u}^n$$

—

코드 구조 설명 행렬 또는 벡터 $\mathbf{b}$ 는 각 시간 단계에서 공간 격자의 온도를 저장한다.

```
b = np.zeros((nt, N))
```

```
for j in range(nt):
    b[j, :] = u.copy()
```

여기서

- nt : 시간 단계 수
- N : 공간 격자 수
- $b[j, :]$: 시간 t_j 에서의 온도 분포

—

실행 입력 예시

알고리즘 선택 (1:명시적, 2:암시적, 3:Crank–Nicolson)

공간 분할 수 N

시간 분할 수 nt

그래프 출력 시간 단계 수

—

결과 시각화 온도 분포는 시간에 따라 다음과 같이 표현된다.

$$u(x, t_0), \quad u(x, t_1), \quad \dots, \quad u(x, t_{nt})$$

Crank–Nicolson 방법은 시간이 증가함에 따라 안정적으로 정상 상태로 수렴한다.

핵심 정리

- 비정상 문제는 시간 전진 계산 수행
- Explicit은 계산이 단순하지만 조건부 안정
- Implicit과 Crank–Nicolson은 선형 시스템 풀이 필요
- CN 방법은 정확도와 안정성 균형이 우수

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter11/section_11_02_08_1d_unsteady_heat_fdm.py

11.3 연습문제